

SUJET INDIVIDUEL ET AUTONOME

Gires Varel TIENTCHEU KAMENI

MATRiCE — Rattrapage WEB2

Modules à réaliser : F2, F3.

Réalisez uniquement les deux modules de ce document. Ils sont indépendants.

UE concernée	Module	Compétence à valider
UE01 — Développement Fullstack	F2	Tests front (Jest, Vitest)
UE01 — Développement Fullstack	F3	UI Libraries (Tailwind, MUI...)

Votre mission

MATRiCE organise des séances de formation par date, demi-journée, groupe et formateur. Vous intervenez sur les compétences indiquées ci-dessus, à partir des données fictives et du code inclus dans ce PDF.

Consignes générales

- Travail individuel. Aucun accès à l'application d'origine, dépôt privé ou fichier pédagogique séparé n'est nécessaire.
- Vous pouvez installer les bibliothèques, runtimes et outils nécessaires.
- L'IA est autorisée et doit être déclarée. Vous devez pouvoir expliquer personnellement tout code ou configuration remis.
- Les captures et traces complètent les preuves ; elles ne remplacent pas une exécution reproductible.

Structure du rendu

Un dossier comprenant README.md, JUSTIFICATIONS.md, SOURCES_IA.md, modules/F2/, modules/F3/ et preuves/.

- **README** : versions, dépendances, installation, commandes exactes de lancement et de tests, ports, variables factices et limites.
- **JUSTIFICATIONS** : pour chaque module, problème → choix → alternative → fichiers/fonctions concernés → explication personnelle → exemple entrée/sortie → test/preuve → limite.
- **SOURCES_IA** : outils et autres sources, usages, fichiers concernés, requêtes représentatives, adaptations et vérifications. Déclarez aussi une absence d'usage d'IA.

Ne livrez ni node_modules, ni secrets réels, ni données personnelles.

Avant remise : vérifiez les commandes, l'autonomie des deux modules, les preuves demandées et vos explications personnelles.

La date et le mode de remise sont communiqués par l'enseignant.

UE01 — DÉVELOPPEMENT FULLSTACK

F2 — Tests front

Votre objectif est de tester un composant React existant et de corriger ses défauts de manière ciblée. Le code volontairement imparfait et les données sont fournis à la page suivante.

Travail demandé

1. Recréez `PlanningList.jsx` et les données. Conservez une copie du composant avant correction.
2. Préparez un tableau **scénario / entrée / attente / risque couvert**.
3. Avec **Jest ou Vitest** et un outil de test React, écrivez au moins les six tests ci-dessous. Pilotez les réponses de `loadSessions` par des doublures ou promesses contrôlées.
4. Montrez **au moins deux tests rouges avant correction**, puis effectuez les corrections minimales et montrez leur passage au vert.

Scénarios à couvrir

Scénario	Comportement attendu
Chargement	Tant que la promesse est en attente, un état de chargement est perceptible.
Succès	Après résolution, les titres des séances reçues sont affichés et le chargement disparaît.
Filtre A	La sélection A demande le bon groupe et affiche les séances A et Promotion, sans les séances B.
Résultat vide	Une réponse <code>[]</code> produit un message explicite, sans ancien résultat affiché.
Erreur puis nouvelle tentative	Un rejet produit une erreur visible. Une action permet de relancer la même demande et de retrouver les résultats.
Réponses dans le désordre	Après deux demandes successives, la réponse tardive de la première ne remplace pas le résultat de la plus récente.

Accessibilité : vérifiez aussi le nom accessible « Groupe » et l'utilisation du filtre au clavier. Ce contrôle peut être intégré à l'un des tests.

Livrables et preuves

- Composant initial et corrigé, tests, configuration, dépendances verrouillées et commandes de test non interactives.
- Tableau des scénarios, traces avant/après identifiables et explication du risque couvert par chaque correction.
- Limites de la stratégie. Les tests doivent observer le comportement rendu ; ne masquez pas les promesses rejetées. Snapshots et couverture seuls ne suffisent pas.

Périmètre : F1 n'est pas requis. Il n'est pas demandé de développer une application complète, un backend ou une base de données.

RESSOURCES INCLUSES — FRONT

Données et composant de départ

Recréez un tableau sessions d'objets avec les clés exactes des deux tableaux. Rapprochez les lignes par id. Toutes les valeurs sont des chaînes, sauf null (valeur JSON).

id	date	period	group	mode
s01	2026-10-19	am	A	DG
s02	2026-10-19	am	B	DG
s03	2026-10-19	pm	Promotion	CE
s04	2026-10-20	am	A	DG
s05	2026-10-20	am	B	DG
s06	2026-10-20	pm	Promotion	AUTO

id	title	domain	teacherId	status
s01	React composants	web	t1	confirmed
s02	React événements	web	t2	confirmed
s03	Données et SQL	data	t1	confirmed
s04	Authentification	cyber	t2	proposed
s05	Revue de projet	projet	t3	proposed
s06	Travail autonome	projet	null	proposed

Règles : A inclut A + Promotion ; B inclut B + Promotion ; « Promotion » ne conserve que Promotion ; « all » conserve tout. AUTO impose teacherId: null et status: "proposed". Une confirmation exige un formateur. Formateurs fictifs : t1 Camille Exemple, t2 Alex Démonstration, t3 Sam Fictif.

Contrat : loadSessions({ group }) renvoie une promesse du tableau filtré. Pour l'usage local, utilisez le filtre ci-dessous dans une fonction async. Dans les tests, simulez également délai, rejet, réponse vide et ordre des réponses.

```
return sessions.filter(s => group === 'all' || s.group === group ||
  (['A', 'B'].includes(group) && s.group === 'Promotion'));
```

PlanningList.jsx — conserver la version initiale

```
import { useEffect, useState } from 'react';

// Point de départ F2 volontairement imparfait.
export default function PlanningList({ loadSessions }) {
  const [group, setGroup] = useState('all');
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(false);
  useEffect(() => {
    setLoading(true);
    loadSessions({ group }).then(result => {
      setItems(result);
      setLoading(false);
    });
  }, [group, loadSessions]);
  return <section>
    <h1>Planning</h1>
    <select aria-label="Groupe" value={group} onChange={e => setGroup(e.target.value)}>
      <option value="all">Tous</option><option value="A">Groupe A</option>
      <option value="B">Groupe B</option><option value="Promotion">Promotion</option>
    </select>
    {loading ? <p role="status">Chargement...</p> : <ul>{items.map(s => <li key={s.id}>{s.title}</li>)}</ul>}
  </section>;
}
```

UE01 — DÉVELOPPEMENT FULLSTACK

F3 — Bibliothèques UI

Réalisez une vue simplifiée du planning avec **React et une bibliothèque UI ou utilitaire** (Tailwind, MUI ou équivalent). Les six séances et les règles de filtrage figurent à la page précédente.

Autonomie : vous pouvez utiliser directement ces données en mémoire. Le composant corrigé en F2 n'est pas un prérequis de F3.

Travail demandé

1. Présentez les séances sous forme de **cartes ou de liste** avec une hiérarchie lisible.
2. Ajoutez un **filtre de groupe** : Tous, A, B et Promotion. A et B incluent les séances communes à Promotion.
3. Affichez des **badges de domaine et de statut**. Le statut doit être compréhensible autrement que par sa couleur, par exemple grâce à un libellé.
4. Permettez d'ouvrir et de fermer le **détail d'une séance**. Préservez la navigation au clavier, les noms accessibles et la gestion du focus, y compris son retour à la fermeture.
5. Prévoyez un **état vide**. Le jeu fourni n'en produit pas avec le seul filtre de groupe : documentez un scénario de démonstration avec une liste vide.
6. Vérifiez la lisibilité et les débordements à **360 px et 1280 px** de largeur.

Trois décisions à justifier

Décision	Explication attendue
Hiérarchie carte / détail	Ce qui apparaît immédiatement et ce qui est réservé au détail.
Lisibilité des statuts	Libellés, couleurs, contraste et compréhension sans la couleur seule.
Accès aux actions	Repérage, noms accessibles, clavier et focus.

Livrables et preuves

- Code, version de la bibliothèque, dépendances et commande de lancement.
- Captures à 360 px et 1280 px ; scénario reproductible montrant l'état vide.
- Protocole clavier : accès au filtre, ouverture du détail, déplacement du focus et fermeture avec retour à l'élément déclencheur.
- Mesure de contraste avec les couleurs vérifiées, le résultat et l'outil utilisé.
- Justification des trois décisions ci-dessus et explication des composants réutilisables.

Périmètre : aucune reproduction complète de MATRiCE, aucun backend ni base de données. F3 n'impose pas de framework de test ; les tests front demandés pour F2 restent à fournir dans ce module distinct.